SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	T. Bhanu Pavan Varma	**Reg. No.:**	192425380
Section / Batch:	B. Tech AI&ML (2024-2028)	**Date:**	

Code of Conduct

I T. Bhanu Pavan Varma(192425380) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Design an algorithm to perform Reference Resolution by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

Inputs, Outputs, Constraints

Input: Tokenized/POS-tagged discourse (list of sentences), list of pronouns to resolve.

Output: Discourse with pronouns replaced by resolved entities; a coreference chain per entity.

Constraints: gender/number agreement, animacy/semantic compatibility (object pronoun "it" must link to an inanimate entity), recency (prefer the most recently mentioned valid candidate), single-antecedent-per-mention.

Pseudocode

```
ALGORITHM ResolveReferences(Discourse D)
Input: D = list of sentences with POS tags and NP chunks
Output: ResolvedDiscourse, CoreferenceChains

1. EntityList <- []                    // all NPs seen so far, in order
2. ChainMap <- empty map               // entity -> list of mentions
3. FOR each sentence S in D:
4.     FOR each token T in S (left to right):
5.         IF T is a Noun Phrase (proper/common noun):
6.             CREATE new entity E with attributes {text, gender, number, animacy}
7.             APPEND E to EntityList
8.             ChainMap[E] <- [T]
9.         ELSE IF T is a Pronoun P:
10.            Candidates <- []
11.            FOR each entity E in REVERSE(EntityList):       // most recent first
12.                IF Gender(E) == Gender(P) AND Number(E) == Number(P):
13.                    IF SemanticCompatible(E, P, context of P):  // animacy + verb-argument fit
14.                        APPEND E to Candidates
15.            IF Candidates is NOT empty:
16.                Antecedent <- Candidates[0]              // recency = first in reverse scan
17.                REPLACE P in text with Antecedent.text       // resolved discourse
18.                APPEND P to ChainMap[Antecedent]
19.            ELSE:
20.                FLAG P as UNRESOLVED
21. RETURN ResolvedDiscourse, ChainMap
END ALGORITHM
```

Application to the Text

Mention	Gender/Number filter	Semantic check	Recency	Resolved to
He	Ravi ✓ (masc, sg), Arun ✓ (masc, sg), library ✗	"borrowed" needs animate agent — both qualify; Ravi is discourse topic	Ravi is discourse topic	Ravi
it	book ✓ (neut), library ✗ (not borrowed item)	"returned it" — patient of return must be borrowed object	book is most recent inanimate NP	book

Resolved discourse:

"Ravi met Arun at the library. Ravi borrowed a book and later returned the book."

Coreference Chains

Chain-1: Ravi → He

Chain-2: Arun (no further mentions)

Chain-3: a book → it

Chain-4: the library (no further mentions)

Validation — How Discourse Context Identifies Antecedents

Syntactic role parallelism: "He" appears as the subject of a new clause right after "Ravi met Arun," and subject-position pronouns preferentially continue the previous subject (Ravi), a well-known discourse-salience heuristic (Centering Theory — the backward-looking center tends to be pronominalized).

Selectional restriction: "borrowed a book" and "returned it" both require an animate agent and inanimate patient respectively, which gender/number alone cannot guarantee — semantic compatibility is essential to rule out "library" as a candidate.

Recency correctly picks "book" over "library" for "it" since "book" was the most recently introduced inanimate entity before the pronoun.

2. Develop an algorithm to analyze Text Coherence and Discourse Structure by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

Inputs, Outputs, Constraints

Input: Sequence of sentences with discourse connectives (if present) and clause-level events.

Output: A discourse relation label for each sentence pair, and a discourse tree.

Constraints: relation set restricted to {Cause-Effect, Sequence, Elaboration, Contrast}; every adjacent pair must receive exactly one relation; connective words are used as primary cues, otherwise fall back to semantic/temporal cues.

Pseudocode

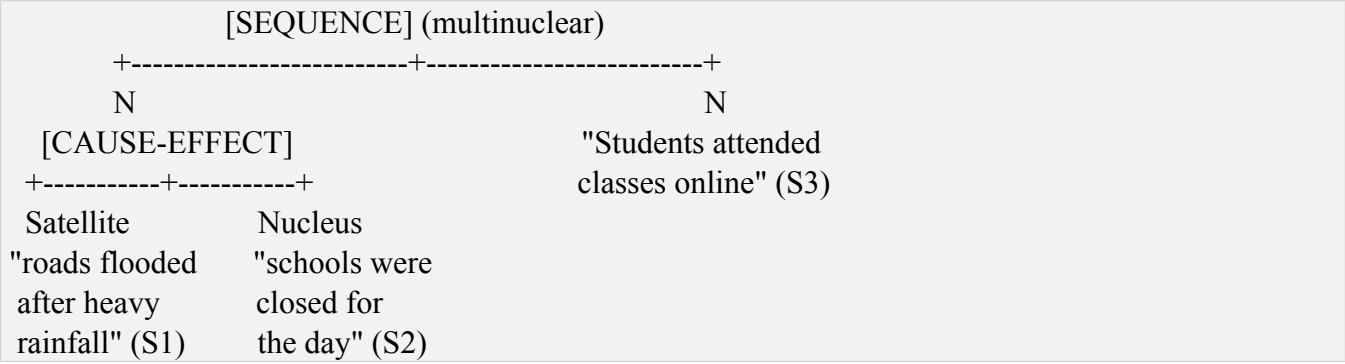
```
ALGORITHM AnalyzeCoherence(Discourse D)
Input: D = ordered list of sentences S1..Sn
Output: RelationList, DiscourseTree

1. ConnectiveMap <- { “therefore”:CauseEffect, “because”:CauseEffect,
                      “then”:Sequence, “after that”:Sequence,
                      “for example”:Elaboration, “which”:Elaboration,
                      “but”:Contrast, “however”:Contrast }
2. RelationList <- []
3. FOR I = 1 to n-1:
4.     Si, Sj <- D[i], D[i+1]
5.     Connective <- EXTRACT explicit connective from Sj (if any)
6.     IF Connective found in ConnectiveMap:
7.         Relation <- ConnectiveMap[Connective]
8.     ELSE:
9.         IF Sj describes an event TEMPORALLY AFTER Si with no logical dependency:
10.            Relation <- Sequence
11.        ELSE IF Sj provides additional detail about an entity/event in Si:
12.            Relation <- Elaboration
13.        ELSE IF Sj’s event is a logical CONSEQUENCE of Si’s event:
14.            Relation <- CauseEffect
15.        ELSE:
16.            Relation <- Sequence     // default fallback
17.    APPEND (Si, Sj, Relation) to RelationList
18. DiscourseTree <- BUILD_RST_TREE(RelationList)   // nucleus-satellite assignment
19. RETURN RelationList, DiscourseTree
END ALGORITHM
```

Application to the Text

Pair	Connective	Relation	Nucleus / Satellite
S1 -> S2	"Therefore"	Cause-Effect	Nucleus = S2 (closure), Satellite = S1 (cause)
S2 -> S3	none (implicit)	Sequence	Both nuclei (multinuclear)

Discourse Tree (RST-style)



Justification — Contribution to Coherence

The Cause-Effect relation (S1->S2) is signaled explicitly by "Therefore," which licenses the reader to interpret flooding as the reason schools closed — this is what makes the discourse locally coherent rather than a list of unrelated facts.

The Sequence relation (S2->S3) links the closure decision to its practical consequence (online classes), maintaining global coherence: the whole passage reads as a single causally/temporally chained narrative (rain -> flood -> closure -> online classes).

Explicit connectives ("Therefore") reduce ambiguity and computational cost in relation-detection, which is why the algorithm checks connectives before falling back to semantic inference.

3. Design an algorithm for a Conversational Agent that identifies Dialogue Acts from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?"
Agent: "Sure, where would you like to travel?"
User: "I want to go to Chennai."
Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

Inputs, Outputs, Constraints

Input: Sequence of utterances with speaker tags.

Output: Dialogue-act label per utterance from {Request, Question, Inform, Confirmation, Action}.

Constraints: classification uses surface form (modal/interrogative structure), speech-act verbs, and dialogue position (turn-taking context).

Pseudocode

```
ALGORITHM ClassifyDialogueAct(Utterance U, PrevContext C)
Input: U = utterance text, C = previous dialogue acts
Output: Act label

1. IF U starts with modal request pattern (“Can you”, “Could you”, “Please”):
2.     RETURN Request
3. ELSE IF U ends with “?” AND asks for missing information (WH-word: where/when/what):
4.     RETURN Question
5. ELSE IF U states a fact/preference about the user (“I want”, “I would like”, “I am”):
6.     RETURN Inform
7. ELSE IF U confirms task completion (past-tense passive: “has been”, “is done”, “confirmed”):
8.     RETURN Confirmation / Action      // Action if it also reports a system-performed task
9. ELSE:
10.    RETURN Statement (default/fallback)
END ALGORITHM

ALGORITHM ProcessConversation(Turns T[1..n])
1. Acts <- []
2. FOR each turn Ti in T:
3.     Act <- ClassifyDialogueAct(Ti.text, Acts)
4.     APPEND Act to Acts
5. RETURN Acts
END ALGORITHM
```

Application to the Conversation

Turn	Speaker	Utterance	Dialogue Act
1	User	"Can you book a train ticket for me?"	Request
2	Agent	"Sure, where would you like to travel?"	Question
3	User	"I want to go to Chennai."	Inform
4	Agent	"Your ticket has been booked."	Confirmation / Action

Dialogue-Act Sequence

```
User:Request --> Agent:Question --> User:Inform --> Agent:Confirmation(Action)
```

Evaluation

The sequence Request -> Question -> Inform -> Confirmation mirrors a canonical slot-filling dialogue pattern: the agent identifies the user's goal (Request), asks a clarifying question to fill a missing slot (destination), receives the Inform that fills the slot, and finally reports task completion (Confirmation/ Action).

Correct act identification lets the agent choose the right response strategy at each turn — a Request should trigger slot-checking logic, a Question expects a slot-value in return, and Inform should trigger a database/booking action once all slots are filled — so accurate classification is essential for correct intent interpretation and task execution.

4. Design an algorithm for Language Generation that converts a semantic representation into a grammatically correct sentence through Surface Realization. Consider the semantic input:

```
Action: Buy
Agent: Student
Object: Book
Tense: Past
```

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

Inputs, Outputs, Constraints

Input: Semantic frame (predicate-argument structure) with tense feature.

Output: Grammatically correct natural-language sentence.

Constraints: correct subject-verb agreement, correct past-tense morphology, correct word order (SVO for English), correct article/determiner insertion.

Pseudocode

```
ALGORITHM GenerateSentence(SemanticFrame F)
Input: F = {Action, Agent, Object, Tense, …}
Output: Sentence (string)

1. // Step 1: Lexical Selection
2. VerbLemma <- LookupLexicon(F.Action)          // “Buy” -> “buy”
3. AgentNP <- DeterminerSelect(F.Agent) + F.Agent  // “The student” / “A student”
4. ObjectNP <- DeterminerSelect(F.Object) + F.Object // “a book”

5. // Step 2: Morphological Realization (Tense/Agreement)
6. IF F.Tense == “Past”:
7.     Verb <- Inflect(VerbLemma, PAST)          // buy -> bought (irregular lookup)
8. ELSE IF F.Tense == “Present” AND IsSingular(F.Agent):
9.     Verb <- Inflect(VerbLemma, PRESENT_3SG)     // buy -> buys
10. ELSE:
11.    Verb <- VerbLemma

12. // Step 3: Sentence Structuring (SVO order for English)
13. Sentence <- AgentNP + “ “ + Verb + “ “ + ObjectNP + “.”

14. // Step 4: Surface Realization / Grammar Check
15. Sentence <- CapitalizeFirstLetter(Sentence)
16. IF NOT PassesGrammarCheck(Sentence):
17.        Sentence <- RepairAgreementErrors(Sentence)
18. RETURN Sentence
END ALGORITHM
```

Application

Step	Result
Lexical selection	Agent -> "The student", Verb lemma -> "buy", Object -> "a book"
Morphological realization	Tense = Past, "buy" is irregular -> "bought"
Sentence structuring	Subject-Verb-Object order: [The student] [bought] [a book]
Surface realization	Capitalization + full stop

Generated sentence: "The student bought a book."

Validation

Lexical selection correctly maps the abstract predicate Buy to its base verb form and inserts appropriate determiners ("The" for the known Agent role, "a" for the newly introduced Object).

Sentence structuring applies English's canonical SVO order, placing Agent before Verb before Object.

Surface realization / grammar check correctly applies the irregular past-tense form "bought" (not "buyed"), verified against an irregular-verb lookup table rather than a blind "+ed" rule.

The final sentence passes subject-verb agreement and produces a complete, grammatical English sentence.

5. Design an algorithm that combines Interlingua-based Machine Translation and Statistical Translation Approaches. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. **Analyze the source sentence.**
2. **Create an Interlingua representation.**
3. **Generate candidate target-language translations.**
4. **Apply statistical scoring to select the most appropriate translation.**
5. **Produce the final translated sentence.**

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

Inputs, Outputs, Constraints

Input: Source-language sentence.

Output: Best-scoring target-language translation.

Constraints: must preserve predicate-argument structure via a language-neutral Interlingua; must generate multiple candidates and rank via statistical scoring before final selection.

Pseudocode

```
ALGORITHM InterlinguaStatisticalMT(SourceSentence Src, TargetLang L)
Input: Src = source sentence, L = target language
Output: BestTranslation

1. // Step 1: Source Analysis
2. ParseTree <- SyntacticParse(Src)
3. PredicateArgs <- SemanticParse(ParseTree)     // play(agent:boy, object:football, tense:present-continuous)

4. // Step 2: Interlingua Representation
5. IR <- BuildInterlingua(PredicateArgs)
6.      IR = < event: PLAY,
               agent: (entity: BOY, number: SG, definiteness: DEF),
               theme: (entity: FOOTBALL),
               aspect: PROGRESSIVE,
               tense: PRESENT >

7. // Step 3: Candidate Generation (Interlingua -> Target)
8. Candidates <- []
9. FOR each generation rule Rk applicable to L:
10.       Ck <- RealizeFromInterlingua(IR, Rk)
11.       APPEND Ck to Candidates

12. // Step 4: Statistical Scoring
13. FOR each candidate C in Candidates:
14.       Score(C) <- w1*LanguageModelProb(C) + w2*TranslationModelProb(C, Src)
15. BestTranslation <- argmax over Candidates of Score(C)

16. RETURN BestTranslation
END ALGORITHM
```

Application to the Sentence (Target Language: Hindi)

1. Source analysis:

Subject = "The boy" (agent), Verb = "is playing" (present progressive), Object = "football" (theme).

2. Interlingua representation:

```
< event: PLAY,
 agent: (entity: BOY, number: SG, definiteness: DEF),
 theme: (entity: FOOTBALL),
 aspect: PROGRESSIVE,
 tense: PRESENT >
```

3. Candidate target-language translations:

C1: लड़का फुटबॉल खेल रहा है। (grammatically correct, natural word order)

C2: फुटबॉल लड़का खेल रहा है। (unnatural word order)

C3: लड़का खेल रहा है फुटबॉल। (broken word order)

4. Statistical scoring:

Candidate	LM score (fluency)	TM score (adequacy)	Combined
C1	High	High	Highest
C2	Low	Medium	Low
C3	Low	Low	Lowest

5. Final translated sentence: लड़का फुटबॉल खेल रहा है। (The boy is playing football.)

The Interlingua stage guarantees that the core meaning (agent, action, theme, tense/aspect) is preserved language-independently before any target text is generated — this prevents meaning drift that pure phrase-based statistical MT can introduce.

The statistical scoring stage handles what Interlingua alone cannot: choosing the most fluent and natural surface realization among several grammatically valid candidates, using corpus-derived probabilities.

Reduced ambiguity: because translation starts from a disambiguated semantic frame, lexical/structural ambiguities in the source are resolved once during parsing rather than repeatedly at the phrase level, improving translation quality.

Trade-off: building accurate Interlingua representations requires deep parsing and is harder to scale to many language pairs than end-to-end statistical/neural MT — the hybrid approach suits cases needing high semantic fidelity over maximum fluency at scale.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____